Translating Out of Static Single Assignment Form

Vugranam C. Sreedhar[1], Roy Dz-Ching Ju[2], David M. Gillies[3], and Vatsa Santhanam[4]

Performance Delivery Laboratory
Hewlett-Packard Company
11000 Wolfe Road
Cupertino, CA 95014, USA

Abstract. Programs represented in Static Single Assignment (SSA) form contain phi instructions (or functions) whose operational semantics are to merge values coming from distinct control flow paths. However, translating phi instructions into native instructions is nontrivial when transformations such as copy propagation and code motion have been performed. In this paper we present a new framework for translating out of SSA form. By appropriately placing copy instructions, we ensure that none of the resources in a phi congruence class interfere. Within our framework, we propose three methods for copy placement. The first method pessimistically places copies for all operands of phi instructions. The second method uses an interference graph to guide copy placement. The third method uses both data flow liveness sets and an interference graph to guide copy placement. We also present a new SSA-based coalescing method that can selectively remove redundant copy instructions with interfering operands. Our experimental results indicate that the third method results in 35% fewer copy instructions than the second method. Compared to the first method, the third method, on average, inserts 89.9% fewer copies during copy placement and runs 15% faster, which are significant reductions in compilation time and space.

1. Introduction

Static Single Assignment (SSA) form is an intermediate representation that facilitates the implementation of powerful program optimizations [7, 12, 13], where each program name is defined exactly once and phi ( ) instructions (or nodes) are inserted at confluent points to merge multiple values into a single name. Phi instructions are not directly supported on current architectures, and hence they must be eliminated prior to final code generation [7]. However, translating out of SSA form is nontrivial when certain transformations, such as copy folding and code motion, have been

[1] Vugranam C. Sreedhar is currently affiliated with IBM T. J. Watson Research Center, and his e-mail address is sreedhar@watson.ibm.com.

[2] The e-mail address of Roy D. C. Ju is royju@cup.hp.com.

[3] David Gillies is currently affiliated with Programmer Productivity Research Center at Microsoft Research, and his e-mail address is dgillies@research.microsoft.com.

[4] The e-mail address of Vatsa Santhanam is vatsa@cup.hp.com.

A. Cortesi, G. Filé (Eds.): SAS'99, LNCS 1694, pp. 194–210, 1999

performed. Most of the previous work on SSA form have concentrated either on efficiently constructing the representation [7, 11], or on proposing new SSA-based optimization algorithms [4, 5, 12].

We are aware of the following published articles related to translating out of SSA form. Cytron et al. [7] proposed a simple algorithm for removing a k-input phi instruction by placing ordinary copy instructions (or assignments) at the end of every control flow predecessor of the basic block containing the phi instruction. Cytron et al. then used Chaitin's coalescing algorithm to reduce the number of copy instructions [3]. The work in [2] showed that Cytron et al.'s algorithm cannot be used to correctly eliminate phi instructions from an SSA representation that has undergone transformations such as copy folding and value numbering. To address this, Briggs et al. [2] proposed an alternative solution for correctly eliminating phi instructions. Briggs et al. exploit the structural properties of both the control flow graph and the SSA graph of a program to detect particular patterns, and use liveness information to guide copy insertions for eliminating phi instructions. Any redundant copies introduced during phi instruction elimination are then eliminated using Chaitin's coalescing algorithm [3]. Pineo and Soffa [10] used interference graph and graph coloring to translate programs out of SSA form for the purpose of symbolic debugging of parallelized code. Leung and George [8] constructed SSA form for programs represented as native machine instructions, including the use of machine dedicated registers. Upon translating out of SSA form, a large number of copy instructions, including many redundant ones, may be inserted to preserve program semantics, and they rely on a coalescing phase in register allocation to remove the redundant copy instructions.

In this paper we present a new framework for leaving SSA form and for eliminating redundant copies. We introduce the notion of a *phi congruence class* to facilitate the removal of phi instructions. Intuitively, a phi congruence class contains a set of resources (or variables) that will be given the same name when we translate out of SSA form. The key intuition behind our method for eliminating phi instructions is to ensure that none of the resources within a phi congruence class interfere among each other. The idea is very similar to coloring-based register allocation problem [3], where if two live ranges interfere they should be given two different physical registers. But if there is only one unused physical register available, then one of the live ranges should be spilled to eliminate the interference. To break the interferences among resources in a phi instruction we introduce "spill code" by placing copy instructions. Another unique aspect of our method is that we don't use any structural properties of either the control flow graph or the SSA graph to guide us in the placement of copy instructions.

We present three different methods of varying sophistication for placing copies. Our first method is closely related to the copy placement algorithm described in [7] except that it correctly eliminates copies even when transformations, such as copy folding and code motion, have been performed on the SSA form of a program. This method does not explicitly use either liveness or interference information to guide copy insertion and placement, and therefore places many more copies than necessary. To reduce the number of copies that are needed to correctly eliminate phi instruction, our second method uses an interference graph to guide copy placement. Although it places fewer copies than the first method, it still places more copies than necessary. To further reduce the number of copies, our third method uses both liveness and interference information to correctly eliminate phi instructions. A unique aspect of